

LAPORAN PRAKTIKUM STRUKTUR DATA

Algoritma Sorting 2

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Dosen Pengampu :

Dr. Wahyudi, M.T

Asisten Praktikum :

M Galid Avaro



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Laporan praktikum ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum Struktur Data yang membahas tentang Algoritma Sorting. Melalui laporan ini penulis dapat memahami materi praktikum secara mendalam dan dapat lebih teliti, teratur, serta memiliki kemampuan yang baik dalam penulisan kode program sesuai kaidah akademik. Sehingga laporan ini dapat menjadi sarana belajar, dokumentasi kegiatan, dan referensi praktikum berikutnya.

Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari isi maupun penyajiannya. Oleh karena itu, saran dan kritik sangat penulis harapkan guna untuk menyempurnakan laporan berikutnya.

Padang, 29 Mei 2026

Aliifah Felda Mufarrihati Salwaa

2511531011

DAFTAR ISI

KATA PENGANTAR.....	i
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum.....	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	3
BAB II PEMBAHASAN	4
2.1. Dasar Teori	4
2.2. Class QuickSort_2511531011	5
2.3. ShellSort_2511531011	6
2.4. MergeSort_2511531011	8
2.5. BubbleSortGUI_2511531011.....	10
2.6. Tugas Pekan 8	14
BAB III KESIMPULAN.....	19
3.1. Kesimpulan.....	19
3.2. Saran	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Praktikum algoritma sorting dilakukan untuk memahami cara kerja proses pengurutan data dalam pemrograman, karena sorting merupakan salah satu teknik dasar yang sangat penting dalam pengolahan data. Dengan mempelajari algoritma seperti Quick Sort, Shell Sort, dan Merge Sort mahasiswa dapat mengetahui bagaimana data dapat diurutkan secara sistematis dari nilai terkecil ke terbesar maupun sebaliknya. Selain itu, praktikum ini membantu mahasiswa memahami logika perbandingan, pertukaran data, serta efisiensi setiap algoritma sorting dalam menyelesaikan suatu permasalahan. Melalui implementasi program dan visualisasi langkah-langkah sorting, mahasiswa diharapkan mampu meningkatkan kemampuan analisis algoritma dan pemrograman secara lebih terstruktur.

1.2 Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami tentang algoritma sorting dalam pemrograman.
2. Mengetahui kelebihan dan kekurangan algoritma sorting dalam program.
3. Dapat membangun sebuah kode program dengan algoritma sorting.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mampu mengimplementasikan algoritma sorting dalam pembuatan program agar lebih efektif dan efisien.
2. Memahami penggunaan algoritma sorting dalam program.

BAB II

PEMBAHASAN

2.1. Dasar Teori

Algoritma sorting adalah metode atau langkah-langkah yang digunakan untuk mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau dari besar ke kecil. Sorting sangat penting dalam pemrograman karena dapat mempermudah proses pencarian, pengelolaan, dan analisis data. Dengan data yang terurut, proses pengolahan informasi menjadi lebih cepat dan efisien. Beberapa contoh algoritma sorting yang akan dipelajari pada praktikum kali ini adalah quick sort, shell sort, dan merge sort.

Quick sort adalah salah satu algoritma sorting yang membagi array menjadi bagian-bagian kecil. Pada pembelajaran struktur data, quick sort yang akan kita gunakan adalah quick sort dengan metode median of three. Metode ini bekerja dengan cara memilih 3 bagian dalam data (low, mid, high) dan kemudian mengubah nilai mid menjadi pivot yang terletak di indeks terakhir array. Metode ini membantu mengurangi worst case dari algoritma quick sort.

Shell sort adalah pengembangan dari insertion sort. Shell sort menggunakan gap untuk membandingkan suatu data, yang mana gap lama-kelamaan akan menjadi 1 dan dapat membandingkan semua elemen array. Penentuan gap dapat dilakukan dengan rumus $n/2^k$ dimana n adalah panjang array dan k adalah increment dari 1. Shell sort ini lebih cepat dibandingkan dengan insertion sort, namun eksekusinya tidak cepat untuk data yang sangat besar.

Merge sort adalah algoritma sorting dengan membagi array menjadi dua bagian. Pembagian dilakukan hingga ukuran 1 data. Kemudian data akan dibandingkan, diurutkan, dan digabungkan kembali. Metode ini akan terus diulangi hingga semua elemen array tergabung kembali. Merge sort merupakan algoritma sorting yang stabil dan cocok untuk data yang besar. Namun, algoritma ini membutuhkan banyak memori tambahan dan implementasinya lebih kompleks dibanding beberapa algoritma lainnya.

2.2. Class QuickSort_2511531011

```
package pekan8_2511531011;

public class QuickSort_2511531011 {
    static void swap_1011(int[] arr_1011, int i, int j) {
        int temp_1011 = arr_1011[i];
        arr_1011[i] = arr_1011[j];
        arr_1011[j] = temp_1011;
    }

    //Metode tambahan untuk mengatur pivot dengan median of three
    static void medianOfThree_1011 (int[] arr_1011, int low_1011, int
high_1011) {
        int mid_1011 = low_1011 + (high_1011-low_1011) /2;

        //Urutkan elemen low, mid, dan high
        if (arr_1011[low_1011] > arr_1011[mid_1011]) {
            swap_1011(arr_1011, low_1011, mid_1011);
        }
        if (arr_1011[low_1011] > arr_1011[high_1011]) {
            swap_1011(arr_1011, low_1011, high_1011);
        }
        if (arr_1011[mid_1011] > arr_1011[high_1011]) {
            swap_1011(arr_1011, mid_1011, high_1011);
        }
        swap_1011(arr_1011, mid_1011, high_1011);
    }

    static int partition_1011(int[] arr_1011, int low_1011, int
high_1011) {
        //Panggil fungsi median of three sebelum menentukan pivot
        medianOfThree_1011(arr_1011, low_1011, high_1011);

        int pivot_1011 = arr_1011[high_1011]; // Sekarang arr[high]
sudah berisi nilai median
        int i = (low_1011 - 1);

        for (int j = low_1011; j<=high_1011-1; j++) {
            //Jika elemen saat ini lebih kecil dari atau sama
dengan pivot
            if (arr_1011[j] < pivot_1011) {
                //Increment indeks elemen yang lebih kecil
                i++;
                swap_1011(arr_1011, i, j);
            }
        }
        swap_1011(arr_1011, i + 1, high_1011);
        return (i + 1);
    }

    static void quickSort_1011(int[] arr_1011, int low_1011, int
high_1011) {
        if (low_1011 < high_1011) {
            int pi_1011 = partition_1011(arr_1011, low_1011,
high_1011);
```

```

        quickSort_1011(arr_1011, low_1011, pi_1011 - 1);
        quickSort_1011(arr_1011, pi_1011 + 1, high_1011);
    }
}

public static void printArr_1011 (int[] arr_1011) {
    for (int i = 0; i < arr_1011.length; i++) {
        System.out.print(arr_1011[i] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int[] arr_1011 = {10, 7, 8, 9, 1, 5};
    int N_1011 = arr_1011.length;
    System.out.print("Data sebelum diurutkan: ");
    printArr_1011(arr_1011);

    quickSort_1011(arr_1011, 0, N_1011 - 1);

    System.out.print("Data terurut quicksort: ");
    printArr_1011(arr_1011);
}
}

```

Kode program 2.1: QuickSort_2511531011

Program ini adalah implementasi quick sort dengan median of three. Disini kita menggunakan beberapa meethod untuk mencapai hasil quick sort yang optimal, diantaranya:

- Method swap: berfungsi untuk menukar elemen
- Method medianOfThree: untuk menentukan pivot dengan nilai median
- Method partition: membagi array (lebih kecil dari pivot dan lebih besar dari pivot)
- Method quickSort: untuk menjalankan semua fungsi agar algoritma quick sort dapat berjalan
- Method main: untuk tes fungsi algoritma dan memberikan contoh

Output:

```

Data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quicksort: 1 5 7 8 9 10

```

2.3. ShellSort_2511531011

```

package pekan8_2511531011;

public class ShellSort_2511531011 {
    public static void shellSort_1011 (int[] A_1011) {
        int n_1011 = A_1011.length;
        int gap_1011 = n_1011/2;
        while (gap_1011 > 0) {
            for (int i = gap_1011; i < n_1011; i++) {
                int temp_1011 = A_1011[i];
                int j = i;
                while (j >= gap_1011 && A_1011[j - gap_1011]
> temp_1011) {
                    A_1011[j] = A_1011 [j-gap_1011];
                    j = j - gap_1011;
                }
                A_1011[j] = temp_1011;
            }
            gap_1011 = gap_1011/2;
        }

        public static void main(String[] args) {
            int[] data_1011 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};

            System.out.print("Sebelum: ");
            printArray_1011(data_1011);

            shellSort_1011(data_1011);

            System.out.print("Sesudah (Shell Sort): ");
            printArray_1011(data_1011);
        }

        public static void printArray_1011(int[] arr_1011) {
            for (int i : arr_1011) System.out.print(i + " ");
            System.out.println();
        }
    }
}

```

Kode program 2.2: ShellSort_2511531011

Program ini merupakan implementasi dari algoritma shell sort. Program ini akan membandingkan suatu elemen dengan gap tertentu. Gap akan terus berubah hingga nilai 1 sehingga semua indeks yang terdapat di dalam array dapat diurutkan seperti metode sorting lainnya. Tetapi dengan metode gap, langkah terakhir untuk mengurutkan seluruh elemen akan menjadi lebih mudah karena beberapa indeks sebelumnya sudah terurut akibat pertukaran dengan gap yang lebih besar.

Output:

```
Sebelum: 3 10 4 6 8 9 7 2 1 5  
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.4. MergeSort_2511531011

```
package pekan8_2511531011;  
  
public class MergeSort_2511531011 {  
    void merge_1011 (int arr_1011[], int l_1011, int m_1011, int  
r_1011) {  
        //Find size of two subarrays to be merged  
        int n1_1011 = m_1011 - l_1011 + 1;  
        int n2_1011 = r_1011 - m_1011;  
        // Create temporary arrays  
        int L_1011[] = new int[n1_1011];  
        int R_1011[] = new int[n2_1011];  
        // Copy data to temp arrays  
        for (int i = 0; i < n1_1011; ++i)  
            L_1011[i] = arr_1011[l_1011 + i];  
        for (int j = 0; j < n2_1011; ++j)  
            R_1011[j] = arr_1011[m_1011 + 1 + j];  
        int i = 0, j = 0;  
  
        // Initial index of merged subarray array  
        int k_1011 = l_1011;  
        while (i < n1_1011 && j < n2_1011) {  
            if (L_1011[i] <= R_1011[j]) {  
                arr_1011[k_1011] = L_1011[i];  
                i++;  
            } else {  
                arr_1011[k_1011] = R_1011[j];  
                j++;  
            }  
            k_1011++;  
        }  
  
        // Copy remaining elemets of L[] if any  
        while (i < n1_1011) {  
            arr_1011[k_1011] = L_1011[i];  
            i++;  
            k_1011++;  
        }  
  
        // Copy remaining element of R[] if any  
        while (j < n2_1011) {  
            arr_1011[k_1011] = R_1011[j];  
            j++;  
            k_1011++;  
        }  
    }  
  
    void sort_1011(int arr_1011[], int l_1011, int r_1011) {  
        if (l_1011 < r_1011) {
```

```

        // Find the middle point
        int m_1011 = (l_1011 + r_1011) / 2;
        // Sort first and second halves
        sort_1011(arr_1011, l_1011, m_1011);
        sort_1011(arr_1011, m_1011 + 1, r_1011);
        // Merge the sorted halves
        merge_1011(arr_1011, l_1011, m_1011, r_1011);
    }
}

// A utility function to print array to size n
static void printArray_1011(int arr_1011[]) {
    int n_1011 = arr_1011.length;
    for (int i = 0; i < n_1011; ++i)
        System.out.print(arr_1011[i] + " ");
    System.out.println();
}

public static void main(String args[]) {
    int arr_1011[] = {12, 11, 13, 5, 6, 7};
    System.out.print("Sebelum diurutkan: ");
    printArray_1011(arr_1011);

    MergeSort_2511531011 ob = new MergeSort_2511531011();
    ob.sort_1011(arr_1011, 0, arr_1011.length - 1);

    System.out.print("\nSesudah terurut menggunakan merge sort:
");
    printArray_1011(arr_1011);
}
}

```

Kode program 2.3: MergeSort_2511531011

Program ini merupakan implementasi dari algoritma sorting merge sort. Disini terdapat beberapa method dengan fungsi masing-masing agar merge sort dapat berfungsi dengan benar, diantaranya:

- Method Merge: untuk menggabungkan dua array setelah dipisahkan
- Method sort: untuk mengurutkan array dan menggabungkannya
- Method printArray: untuk print semua array
- Method main: untuk mencoba menjalankan program merge sort

Output:

```

Sebelum diurutkan: 12 11 13 5 6 7
Sesudah terurut menggunakan merge sort: 5 6 7 11 12 13

```

2.5. BubbleSortGUI_2511531011

```
package pekan8_2511531011;

import java.awt.*;
import javax.swing.*;

import pekan7_2511531011.InsertionGUI_2511531011;

public class BubbleSortGUI_2511531011 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_1011;
    private JLabel[] labelArray_1011;
    private JButton stepButton_1011, resetButton_1011,
setButton_1011;
    private JTextField inputField_1011;
    private JPanel panelArray_1011;
    private JTextArea stepArea_1011;

    private int i = 1, j;
    private boolean sorting_1011 = false;
    private int stepCount_1011 = 1;

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511531011() {
        setTitle("Bubble Sort langkah per langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        //panel input
        JPanel inputPanel_1011 = new JPanel(new FlowLayout());
        inputField_1011 = new JTextField(30);
        setButton_1011 = new JButton("Set Array");
        inputPanel_1011.add(new JLabel ("Masukkan angka (pisahkan
dengan koma): "));
        inputPanel_1011.add(inputField_1011);
        inputPanel_1011.add(setButton_1011);

        //panel array visual
        panelArray_1011 = new JPanel();
        panelArray_1011.setLayout(new FlowLayout());

        //panel kontrol
        JPanel controlPanel_1011 = new JPanel();
        stepButton_1011 = new JButton("Langkah selanjutnya");
        resetButton_1011 = new JButton("Reset");
        stepButton_1011.setEnabled(false);
        controlPanel_1011.add(stepButton_1011);
        controlPanel_1011.add(resetButton_1011);

        //Area text untuk log langkah-langkah
        stepArea_1011 = new JTextArea(8,60);
```

```

        stepArea_1011.setEditable(false);
        stepArea_1011.setFont(new Font("Monospaced", Font.PLAIN,
14));
        JScrollPane scrollPane_1011 = new
JScrollPane(stepArea_1011);

        // tambahkan panel ke frame
        add(inputPanel_1011, BorderLayout.NORTH);
        add(panelArray_1011, BorderLayout.CENTER);
        add(controlPanel_1011, BorderLayout.SOUTH);
        add(scrollPane_1011, BorderLayout.EAST);

        // event set array
        setButton_1011.addActionListener(e ->
setArrayFromInput_1011());

        // event langkah selanjutnya
        stepButton_1011.addActionListener(e -> performStep_1011());

        // event reset
        resetButton_1011.addActionListener(e -> reset_1011());

    }

    private void setArrayFromInput_1011() {
        String text_1011 = inputField_1011.getText().trim();
        if (text_1011.isEmpty()) return;
        String[] parts_1011 = text_1011.split(", ");
        array_1011 = new int [parts_1011.length];
        try {
            for (int k = 0; k < parts_1011.length; k++) {
                array_1011[k] =
Integer.parseInt(parts_1011[k].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya
angka "
                                + "yang dipisahkan dengan koma!",
                                "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
        i = 0;
        j = 0;
        stepCount_1011 = 1;
        sorting_1011 = true;
        stepButton_1011.setEnabled(true);
        stepArea_1011.setText("");
        panelArray_1011.removeAll();
        labelArray_1011 = new JLabel[array_1011.length];
        for (int k = 0; k < array_1011.length; k++) {
            labelArray_1011[k] = new
JLabel(String.valueOf(array_1011[k]));
            labelArray_1011[k].setFont(new Font("Arial",
Font.BOLD, 24));
            labelArray_1011[k].setOpaque(true);
            labelArray_1011[k].setBackground(Color.WHITE);

```

```

        labelArray_1011[k].setBorder(BorderFactory.createLineBorder(Color
.BLACK));
        labelArray_1011[k].setPreferredSize(new
Dimension(50, 50));

        labelArray_1011[k].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_1011.add(labelArray_1011[k]);
    }
    panelArray_1011.revalidate();
    panelArray_1011.repaint();
}

private void performStep_1011() {
    if (!sorting_1011 || i >= array_1011.length - 1) {
        sorting_1011 = false;
        stepButton_1011.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesai!");
        return;
    }
    resetHighLights_1011();
    StringBuilder stepLog_1011 = new StringBuilder();
    labelArray_1011[j].setBackground(Color.CYAN);
    labelArray_1011[j + 1].setBackground(Color.CYAN);
    if (array_1011[j] > array_1011[j + 1]) {
        //swap
        int temp_1011 = array_1011[j];
        array_1011[j] = array_1011[j + 1];
        array_1011[j + 1] = temp_1011;
        labelArray_1011[j].setBackground(Color.RED);
        labelArray_1011[j + 1].setBackground(Color.RED);

        stepLog_1011.append("langkah").append(stepCount_1011).append(":
Menukar elemen ke-")
            .append(j).append(" ").append(array_1011[j +
1]).append(") dengan ke-")
            .append(j + 1).append("
(").append(array_1011[j]).append(")\n");
    } else {

        stepLog_1011.append("langkah").append(stepCount_1011).append(":
Tidak ada pertukaran antara ke-")
            .append(j).append(" dan ke-").append(j +
1).append("\n");
    }
    stepLog_1011.append("Hasil:
").append(arrayToString_1011(array_1011)).append("\n\n");
    stepArea_1011.append(stepLog_1011.toString());
    updateLabels_1011();
    j++;
    if (j >= array_1011.length - i - 1) {
        j = 0;
        i++;
    }
    stepCount_1011++;
    if (i >= array_1011.length - 1) {

```

```

        sorting_1011 = false;
        stepButton_1011.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting
selesail!");
    }
}

private void updateLabels_1011() {
    for (int k = 0; k < array_1011.length; k++) {

        labelArray_1011[k].setText((String.valueOf(array_1011[k])));
    }
}

private void resetHighLights_1011() {
    for (JLabel label_1011 : labelArray_1011) {
        label_1011.setBackground(Color.WHITE);
    }
}

private void reset_1011() {
    inputField_1011.setText("");
    panelArray_1011.removeAll();
    panelArray_1011.revalidate();
    panelArray_1011.repaint();
    stepArea_1011.setText("");
    stepButton_1011.setEnabled(false);
    sorting_1011 = false;
    i = 0;
    j = 0;
    stepCount_1011 = 1;
}

private String arrayToString_1011(int[] arr_1011) {
    StringBuilder sb_1011 = new StringBuilder();
    for (int k = 0; k < arr_1011.length; k++) {
        sb_1011.append(arr_1011[k]);
        if (k < arr_1011.length - 1) sb_1011.append(", ");
    }
    return sb_1011.toString();
}

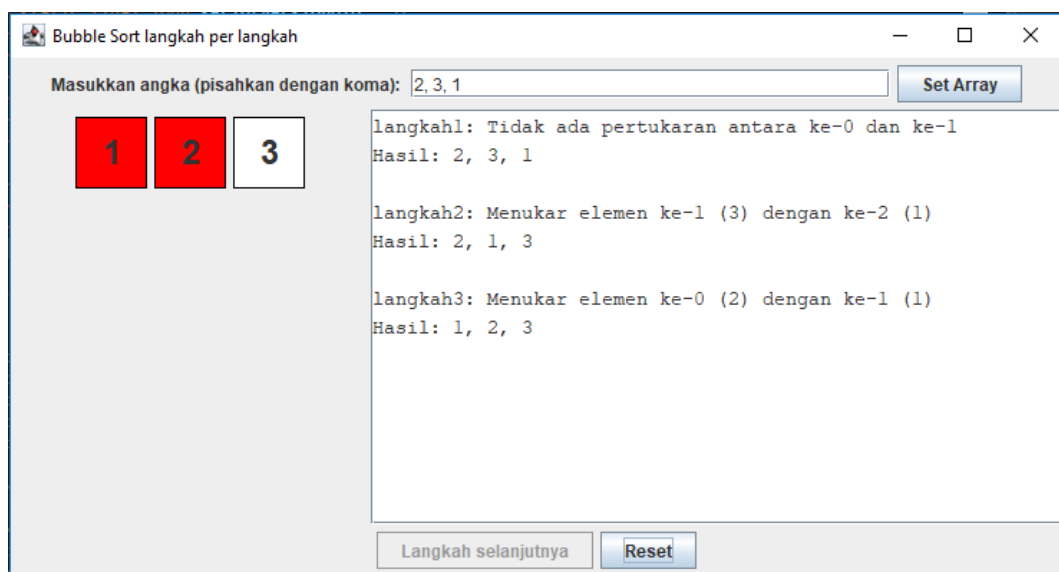
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        BubbleSortGUI_2511531011 gui_1011 = new
BubbleSortGUI_2511531011();
        gui_1011.setVisible(true);
    });
}
}

```

Kode program 2.4: BubbleSortGUI_2511531011

Program ini merupakan contoh implementasi bubble sort dengan GUI. Semua method yang digunakan sama dengan method pada GUI insertion sort sebelumnya. Disini kita dapat melihat bagaimana sistem pengurutan bubble sort dengan visualisasi dan warna. Warna merah pada tampilan angka menunjukkan terjadinya pertukaran dan warna cyan menunjukkan tidak ada pertukaran yang terjadi antar elemen. GUI ini juga memiliki field untuk menjelaskan langkah demi langkah pada pertukaran elemen array.

Contoh output:



2.6. Tugas Pekan 8

A. dataLagu_2511531011

```
package pekan8_2511531011;

public class dataLagu_2511531011 {
    //mendefinisikan class node
    String judul_1011;
    String penyanyi_1011;
    int durasi_1011;

    //konstruktor
    public dataLagu_2511531011 (String judul_1011,
String penyanyi_1011, int durasi_1011) {
        this.judul_1011 = judul_1011;
        this.penyanyi_1011 = penyanyi_1011;
        this.durasi_1011 = durasi_1011;
    }
}
```

```

        //method getter
        public String getJudul_1011() { return judul_1011; }
//get judul
        public String getPenyanyi_1011() { return
penyanyi_1011; } //get penyanyi
        public int getdurasi_1011() { return durasi_1011; }
//get durasi

        //method setter
        public void setJudul_1011(String judul_1011) {
this.judul_1011 = judul_1011; } //set judul
        public void setPenyanyi_1011(String penyanyi_1011) {
this.penyanyi_1011 = penyanyi_1011; } //set penyanyi
        public void setDurasi_1011(int durasi_1011) {
this.durasi_1011 = durasi_1011; } //set durasi

        //untuk cetak data lagu
        @Override
        public String toString () {
            return "Judul Lagu: " + judul_1011 + " - " +
penyanyi_1011 + " Durasi: " + durasi_1011;
        }
    }
}

```

Kode Program 2.5: dataLagu_2511531011

Pada kode program ini kita akan membuat konstruktor untuk algoritma sorting lagu dengan atribut judul, penyanyi, dan durasi. Disini kita juga membuat method setter dan getter untuk setiap atribut yang ada. Kita juga menggunakan override untuk mencetak data lagu.

B. Sorting_2511531011

```

package pekan8_2511531011;

import java.util.Scanner;

public class Sorting_2511531011 {
    Scanner input_1011 = new Scanner(System.in);

    //array yang menyimpan maksimal 20 data
    static dataLagu_2511531011[] dataLagu_1011 = new
dataLagu_2511531011[20];

    //method inputData
    public static dataLagu_2511531011 inputData_1011 (Scanner
input_1011) {
        System.out.print("Judul Lagu: ");
        String judul_1011 = input_1011.nextLine();
        System.out.print("Penyanyi Lagu: ");
    }
}

```

```

        String penyanyi_1011 = input_1011.nextLine();
        System.out.print("Durasi Lagu: ");
        int durasi_1011 = input_1011.nextInt();

        input_1011.nextLine();

        return new dataLagu_2511531011(judul_1011, penyanyi_1011,
durasi_1011);
    }

    static void swap_1011(dataLagu_2511531011[] dataLagu_1011, int i,
int j) {
        dataLagu_2511531011 temp_1011 = dataLagu_1011[i];
        dataLagu_1011[i] = dataLagu_1011[j];
        dataLagu_1011[j] = temp_1011;
    }

    //Metode tambahan untuk mengatur pivot dengan median of three
    static void medianOfThree_1011 (dataLagu_2511531011[]
dataLagu_1011, int low_1011, int high_1011) {
        int mid_1011 = low_1011 + (high_1011-low_1011) /2;

        //Urutkan elemen low, mid, dan high
        if (dataLagu_1011[low_1011].getdurasi_1011() >
dataLagu_1011[mid_1011].getdurasi_1011()) {
            swap_1011(dataLagu_1011, low_1011, mid_1011);
        }
        if (dataLagu_1011[low_1011].getdurasi_1011() >
dataLagu_1011[high_1011].getdurasi_1011()) {
            swap_1011(dataLagu_1011, low_1011, high_1011);
        }
        if (dataLagu_1011[mid_1011].getdurasi_1011() >
dataLagu_1011[high_1011].getdurasi_1011()) {
            swap_1011(dataLagu_1011, mid_1011, high_1011);
        }
        swap_1011(dataLagu_1011, mid_1011, high_1011);
    }

    static int partition_1011(dataLagu_2511531011[] dataLagu_1011,
int low_1011, int high_1011) {
        //Panggil fungsi median of three sebelum menentukan pivot
        medianOfThree_1011(dataLagu_1011, low_1011, high_1011);

        dataLagu_2511531011 pivot_1011 = dataLagu_1011[high_1011];
        // Sekarang arr[high] sudah berisi nilai median
        int i = (low_1011 - 1);

        for (int j = low_1011; j<=high_1011-1; j++) {
            //Jika elemen saat ini lebih kecil dari atau sama
dengan pivot
            if (dataLagu_1011[j].getdurasi_1011() <
pivot_1011.getdurasi_1011()) {
                //Increment indeks elemen yang lebih kecil
                i++;
                swap_1011(dataLagu_1011, i, j);
            }
        }
    }

```

```

    }
    swap_1011(dataLagu_1011, i + 1, high_1011);
    return (i + 1);
}

//method sorting (quickSort)
static void quickSort_1011(dataLagu_2511531011[] dataLagu_1011,
int low_1011, int high_1011) {
    if (low_1011 < high_1011) {
        int pi_1011 = partition_1011(dataLagu_1011,
low_1011, high_1011);
        quickSort_1011(dataLagu_1011, low_1011, pi_1011 -
1);
        quickSort_1011(dataLagu_1011, pi_1011 + 1,
high_1011);
    }
}

//method tampilData
public static void tampilData_1011 (dataLagu_2511531011[]
dataLagu_1011, int jumlah_1011) {
    for (int i = 0; i < jumlah_1011; i++) {
        System.out.println((i+1) + ". " + dataLagu_1011[i]);
    }
}

public static void main(String[] args) {
    Scanner input_1011 = new Scanner(System.in);
    int jumlah_1011;
    System.out.println("=== Sorting Playlist NIM: 2511531011
===");

    //validasi untuk minimal 7 lagu
    do {
        System.out.print("Jumlah lagu(Minimal 7): ");
        jumlah_1011 = input_1011.nextInt();
        input_1011.nextLine();
        if (jumlah_1011 < 7) System.out.println("Minimal
harus 7 lagu!");
    } while (jumlah_1011 < 7);

    for(int i = 0; i < jumlah_1011; i++) {
        System.out.println("\nData lagu ke-" + (i+1));
        dataLagu_1011[i] = inputData_1011(input_1011);
    }

    System.out.println("\nData sebelum diurutkan: ");
    tampilData_1011(dataLagu_1011, jumlah_1011);

    quickSort_1011(dataLagu_1011, 0, jumlah_1011 - 1);

    System.out.println("\n\nData terurut quicksort(durasi asc):
");
    tampilData_1011(dataLagu_1011, jumlah_1011);
}

```

```
}
```

Kode program 2.6: Sorting_2511531011

Pada program ini saya menggunakan algoritma sorting yakni quick sort. Penggunaan quick sort ini lebih efisien dibandingkan dengan algoritma lainnya. Quick sort termasuk cepat untuk data yang banyak, mengurangi penggunaan memori, dan sangat cocok untuk data yang tersimpan di dalam array. Walaupun quick sort memiliki kemungkinan worst case yang cukup buruk, tetapi dengan metode median of three, kita dapat mengurangi kemungkinan worst case pada quick sort ini.

Quick sort dianggap lebih cepat untuk data yang besar dibandingkan dengan algoritma sorting lainnya. Hal ini dikarenakan quick sort memiliki kompleksitas rata-rata $O(n \log n)$. Namun, quick sort juga memiliki worst case yang sangat buruk yakni $O(n^2)$. Oleh karena itu, jika kita ingin menggunakan quick sort, sebaiknya gunakan metode median of three seperti pada tugas. Kita perlu membuat method swap, median of three, dan partition untuk menjadikan algoritma quick sort berjalan dengan baik dan benar.

Output:

```
=== Sorting Playlist NIM: 2511531011 ===
Jumlah lagu(Minimal 7): 7

Data lagu ke-1
Judul Lagu: lucky
Penyanyi Lagu: exo
Durasi Lagu: 345

Data lagu ke-2
Judul Lagu: kokobop
Penyanyi Lagu: exo
Durasi Lagu: 454

Data lagu ke-3
Judul Lagu: ruby
Penyanyi Lagu: woozi
Durasi Lagu: 231

Data lagu ke-4
Judul Lagu: holiday
Penyanyi Lagu: svt
Durasi Lagu: 344

Data lagu ke-5
Judul Lagu: super
Penyanyi Lagu: svt
Durasi Lagu: 435

Data lagu ke-6
Judul Lagu: fighting
Penyanyi Lagu: bss
Durasi Lagu: 554

Data lagu ke-7
Judul Lagu: pretty u
Penyanyi Lagu: svt
Durasi Lagu: 555
```

```
Data sebelum diurutkan:
1. Judul Lagu: lucky - exo Durasi: 345
2. Judul Lagu: kokobop - exo Durasi: 454
3. Judul Lagu: ruby - woozi Durasi: 231
4. Judul Lagu: holiday - svt Durasi: 344
5. Judul Lagu: super - svt Durasi: 435
6. Judul Lagu: fighting - bss Durasi: 554
7. Judul Lagu: pretty u - svt Durasi: 555

Data terurut quicksort(durasi asc):
1. Judul Lagu: ruby - woozi Durasi: 231
2. Judul Lagu: holiday - svt Durasi: 344
3. Judul Lagu: lucky - exo Durasi: 345
4. Judul Lagu: super - svt Durasi: 435
5. Judul Lagu: kokobop - exo Durasi: 454
6. Judul Lagu: fighting - bss Durasi: 554
7. Judul Lagu: pretty u - svt Durasi: 555
```

BAB III

KESIMPULAN

3.1. Kesimpulan

Penggunaan algoritma sorting lebih memudahkan kita dalam mengurutkan sebuah data dari list. Algoritma sorting ini juga memiliki kelebihan dan kekurangannya masing-masing jika diaplikasikan dalam sebuah program. Oleh karena itu, dalam praktikum kali ini kita mengetahui tentang macam-macam algoritma sorting dan bagaimana penerapannya dalam pemrograman.

3.2. Saran

Karena banyaknya jenis dari algoritma sorting, mahasiswa hendaknya terus memahami dan mempelajari bagaimana penggunaan dari algoritma sorting. Tidak hanya melalui praktikum, mahasiswa juga dapat melakukan latihan melalui tugas maupun proyek pribadi dalam penggunaan algoritma sorting. Dari sini diharapkan juga mahasiswa mampu memahami sebagian algoritma sorting dan penerapannya pada program dan GUI.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, dan C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA: MIT Press, 2009.
- [2] Wahyudi, *Struktur Data dan Algoritma*, Padang: Informatika, 2026